COMSATS University Islamabad (Lahore Campus)

Department of Computer Engineering

Implementation Guide

Member B — Edge / CV Development

Detailed Sprint Breakdown & Approach

Companion: `member_b_sprint_plan.tex` · Master: `centralized_sprint_plan.tex`

Spring–Fall 2026

Contents

Chapter 1

Introduction

1.1 Purpose

This guide explains **how to build the CV track** week by week: dataset, training, compliance engine, and the handoff dict Member A consumes. It complements:

- `.sprint/member_b_sprint_plan.tex`
- `finalization/DETECTION_CLASS_SCOPE.md` — supervisor class list
- `.prerequisite/MEMBER_B_PREREQUISITES.md`

1.2 Your boundary

You deliver **working CV code + handoff dict**. Member A handles MQTT, Jetson deploy, and the platform. Your code lives only in `edge/inference/` and `edge/compliance_engine/`.

1.3 Three-track pipeline (always keep this in mind)

Track	Detects	Feeds compliance engine
Worker Detection	`worker` boxes	Positions; IoU anchor
PPE Detection	`helmet`, `vest`	Associated to workers
Hazard Detection	`fire`, `smoke`, `spill`	Zone-level events

1.4 General workflow every week

1. Jira tickets from Sprint backlog (label `member-b`)
2. Work on feature branch under `edge/`
3. Push dataset labels / small files; large weights via Drive or Git LFS (agree with A)
4. Weekly sync: share handoff JSON samples when asked
5. Sunday sprint log with training metrics screenshot

Tip

You can progress **fully on laptop** until Sprint 4 — no Jetson needed. Webcam inference is enough until Member A deploys.

Chapter 2

Sprint 1 (Weeks 1–4): Dataset & Handoff Spec

2.1 Sprint goal

Dev environment ready; annotation pipeline running; 200+ labeled images; handoff interface documented. Align with Member A at **M0**.

How to Approach This Sprint

Order of work:

1. Environment + GPU check (Day 1–3)
2. Dataset sources identified (Week 1)
3. Annotation tool + guidelines doc (Week 2)
4. First 200 labels (Week 3)
5. Handoff spec + pretrained demo (Week 4)

Do not write MQTT or touch `transport/`.

2.1.1 Week 1–2: Environment & data sources

Step: Setup

```
python -m venv venv && source venv/bin/activate
pip install ultralytics opencv-python torch torchvision
python -c "import torch; print(torch.cuda.is_available())"
```

Clone repo; branch `feature/b-edge-setup`; first PR adding `edge/inference/README.md`.

Step: Dataset strategy

Combine sources for 6 classes:

- Public: Roboflow PPE datasets, Kaggle safety sets
- Custom: campus/lab photos for your environment
- Balance: ensure fire/smoke/spill not under-represented (common mistake)

Document sources and licenses in `edge/inference/DATASET.md`.

Step: Annotation guidelines

One `.txt` per image (YOLO format). Rules:

- Tight boxes; include partially visible helmets
- `worker` = full body where possible
- Hazards may appear without workers in frame
- Same image reviewed by consistent rules (write them down)

2.1.2 Week 3–4: Labels & handoff spec

Step: Handoff interface (critical)

Document in `edge/README.md`:

```
def get_compliance_event(frame, zone: str) -> dict | None:
    """
    Returns None if no event this frame.
    Keys: event, zone, worker_id, compliance_score,
          confidence, detections, image_path
    """
```

Map each key to Member A's `contracts/events.py` — meet Week 4 to sign off.

Step: Pretrained demo

Run YOLOv8n COCO-pretrained on sample images; save annotated outputs to `edge/samples/`. Proves pipeline before custom training finishes.

Verification Checklist

- ☐ 200+ images labeled (all 6 classes present)
- ☐ `data.yaml` draft with class names
- ☐ Handoff spec reviewed by Member A (M0)
- ☐ Pretrained inference runs on webcam or video file

Collaboration Touchpoint

Week 4: Attend contract meeting. Provide 3 sample handoff JSON files (helmet_missing, compliant, fire). You do not need to understand MQTT — only field names match.

Common Pitfalls

- Training on 6 classes before enough hazard images — fire/smoke mAP will collapse
- Inconsistent labels between team members — write guidelines early
- Putting MQTT publish in inference loop — belongs to Member A

Chapter 3

Sprint 2 (Weeks 5–8): Training & Compliance Prototype

3.1 Sprint goal

500+ images; YOLOv8n training underway; IoU association + compliance prototype outputting handoff dict to console. Target val mAP ≥ 0.60.

How to Approach This Sprint

Order of work:

1. Expand dataset to 500+ (ongoing Week 5–6)
2. First training run YOLOv8n 640px (Week 5–6)
3. IoU + worker tracking stub (Week 6)
4. Compliance engine module (Week 7–8)
5. Webcam demo printing handoff dict (Week 8)

3.1.1 Week 5–6: Training

Step: Train YOLOv8n

```
yolo detect train data=data.yaml model=yolov8n.pt
  epochs=100 imgsz=640 batch=16 patience=20
```

Monitor `mAP50` on val set. Save best `weights/best.pt`. Log curves screenshot for sprint log.

Step: IoU association

Implement pure Python (no ML):

1. For each worker box, find helmet/vest boxes with IoU > threshold (start 0.3)
2. Assign stable `worker_id` via centroid distance across frames

3. Unit test: synthetic boxes → expected associations

3.1.2 Week 7–8: Compliance engine

Step: Compliance logic

File: `edge/compliance_engine/engine.py`

- Load zone rules from YAML (helmet + vest required per zone)
- Missing associated helmet → `helmet_missing`
- Rolling 5-frame score per worker
- Debounce: same worker + event within 60s → suppress
- Hazard classes: if fire/smoke/spill detected → immediate event (no worker IoU)

Step: Evidence capture

On violation: save frame to `evidence/{session}/frame_NNN.jpg`; set `image_path` in dict. Member A will upload to MinIO later — path string must be consistent.

Step: Console demo

Main loop: webcam → infer → compliance → `print(json.dumps(event))`. Record 30s video for sprint review.

Verification Checklist

- ☐ Val mAP@0.5 $\geq$ 0.60 (improving)
- ☐ IoU tests pass
- ☐ Debounce prevents spam in console
- ☐ Sample handoff JSON files delivered to A (Week 8 integration)

Chapter 4

Sprint 3 (Weeks 9–12): Model Finalization & PR Merge

4.1 Sprint goal

1000+ images; mAP@0.5 $\geq$ 0.70; ONNX export; PRs merged. Member A handoff harness passes. **M2**.

How to Approach This Sprint

Focus order:

1. Fix weakest classes first (inspect confusion matrix)
2. Retrain with augmentation (flip, brightness — not excessive blur)
3. Freeze weights; export ONNX
4. Clean PRs with README + requirements

Optional gloves/boots only if core 6 classes already $\geq$ 0.70.

4.1.1 Week 9–10: Reach mAP target

Step: Per-class analysis

Generate confusion matrix; if `smoke` confused with `fire`, add more distinct smoke images. Document in `edge/inference/EVAL.md`:

- Overall mAP@0.5, precision, recall
- Per-class AP
- Test set size and split method

Step: Hazard track validation

Test video clips: spill on floor, smoke-like steam, small fire images. Hazards must emit events without worker in frame.

4.1.2 Week 11–12: Export & merge

Step: Export for Member A

```
yolo export model=best.pt format=onnx imgsz=640
```

Package: `best.pt`, `best.onnx`, `requirements.txt`, `EVAL.md`, `README.md` setup steps. Share via Drive link in Jira ticket.

Step: Pull requests

- PR1: `edge/inference/` (model loader, inference loop)
- PR2: `edge/compliance_engine/` (rules, IoU, debounce, handoff function)
- Respond to A's review within 48h

No files outside `edge/`.

Verification Checklist

- ☐ mAP@0.5 $\geq$ 0.70 on held-out test set
- ☐ EVAL.md complete
- ☐ PRs merged to `main`
- ☐ Member A confirms harness converts your dict $\rightarrow$ MQTT $\rightarrow$ DB

Collaboration Touchpoint

Week 12: Hand off model package. Answer harness failures quickly — usually field name mismatch or wrong `event` enum string.

Chapter 5

Sprint 4 (Weeks 13–16): Deploy Support

5.1 Sprint goal

Fix Jetson-side issues reported by Member A; stable FPS and handoff on edge camera. **M3**.

How to Approach This Sprint

You **do not SSH to Jetson** unless A asks you to watch. Your job: fix code from Jira bugs, re-export ONNX, tune confidence thresholds from A's feedback logs.

Step: Threshold tuning

If false positives on vest: raise confidence threshold in config YAML. If missed helmets: lower threshold or add hard-hat images similar to Jetson camera angle.

Step: FPS optimization hints for A's deploy

Document in README: use YOLOv8n only; 640 input; TensorRT export command; expected FPS range on Jetson Nano 2GB (>15 target).

Step: Presentation prep

5-minute CV slides: three-track diagram, sample detections, mAP table, handoff boundary diagram, demo video clip (laptop is fine).

Verification Checklist

- ☐ Zero open `blocker` Jira tickets from Jetson testing
- ☐ Helmet-off test works when A runs on Jetson
- ☐ CV slides draft ready

Collaboration Touchpoint

Week 16 integration: Stand by on call. Explain IoU/debounce if supervisor asks. A operates hardware.

Chapter 6

Sprint 5 (Weeks 17–20): Documentation & Viva

6.1 Sprint goal

FYP report CV chapters; viva-ready explanations; joint demo. **M4**.

How to Approach This Sprint

Shift from coding to **explaining**. You must defend every design choice: why 6 classes, why IoU 0.3, why YOLOv8n, why debounce 60s.

Step: Report sections

- Dataset: sources, count, annotation examples
- Training: hyperparameters, hardware, duration
- Evaluation: EVAL.md tables + confusion matrix figure
- Compliance engine: IoU diagram, debounce flowchart
- Integration: handoff dict; boundary with Member A's MQTT (diagram)

Use LaTeX for report; Member A handles platform chapters.

Step: Viva preparation

Practice answers for:

- Walk through one frame: detect → associate → rule → event
- Why not detect goggles/forklift? (supervisor scope)
- What happens after your dict leaves the edge? (high-level platform — A's part)
- Limitations: lighting, occlusion, class confusion

Step: Demo role

Present CV slides (2 min of 12 min demo). A runs live Jetson. rehearse Q&A handoff between you two.

Verification Checklist

- ☐ Report CV sections submitted
- ☐ Can explain every function in merged PRs
- ☐ Joint demo rehearsed twice
- ☐ All `member-b` Jira tickets closed

Chapter 7

Appendix A: Handoff Dict Specification

```
{
  "event": "helmet_missing | vest_missing | fire |
            smoke | spill | compliant",
  "zone": "welding_bay | general | chemical_storage",
  "worker_id": 7,
  "compliance_score": 78,
  "confidence": 0.92,
  "detections": [
    {"class": "helmet", "bbox": [x,y,w,h], "confidence": 0.94}
  ],
  "image_path": "evidence/session_id/frame_001.jpg"
}
```

Member A adds: `type`, `priority`, `timestamp`, `session_id`, MQTT publish.

Chapter 8

Appendix B: Dataset Targets by Sprint

Sprint	Min images	Target mAP@0.5
1	200+ labeled	Pretrained demo only
2	500+	≥ 0.60 val
3	1000+	≥ 0.70 test
4–5	tune only	stable on Jetson

Chapter 9

Appendix C: Troubleshooting

- **Low mAP on one class:** Add 50+ images for that class; check mislabels
- **IoU assigns wrong PPE:** Lower/raise threshold; prefer closest worker centroid
- **Debounce too aggressive:** Reduce window for demo; document production value
- **A says harness fails:** Compare your JSON keys to contract line-by-line
- **CUDA OOM:** Reduce batch size; use `yolov8n` not `s/m`